

LegalEase

AI-Powered Legal Document Generator

Professional Project Documentation
8-Phase Software Development Project Report

Project Focus: AI-assisted generation of structured legal document drafts

Prepared for Academic / Demonstration Use

1. Brainstorming and Ideation

LegalEase is conceived as a web-based application that helps users create structured drafts of common legal documents through guided forms and artificial intelligence. The idea addresses a practical problem: legal documents often require repeated collection of names, dates, addresses, clauses, obligations, and other details. A guided generator can reduce repetitive typing while maintaining a consistent document structure.

The project identifies users such as students, small businesses, individuals, administrators, and legal professionals who may need a starting draft. The system is not intended to replace a qualified lawyer. Instead, it provides an organized draft that can be reviewed, corrected, customized, and approved by an appropriate professional before real-world use.

Core ideas include document-type selection, guided data collection, template management, AI-assisted clause generation, validation, preview, PDF/DOCX export, user authentication, history, and an administrative template-management module. The design emphasizes explainability, privacy, traceability, and safe handling of user-provided information.

The product flow is: user signs in, selects a document type, enters required information, reviews generated clauses, generates a draft, previews it, downloads the document, and optionally saves the project for later editing. The AI layer can suggest language, but deterministic templates and validation rules remain responsible for required fields and document structure.

Suggested System Flow

User → Web Interface → Authentication → Document Form → Validation → Template/AI Service → Output Validation → PDF/DOCX Renderer → Secure Storage → Download

2. Requirement Analysis

Functional requirements define what LegalEase must do. The application should allow registration and login, document-type selection, form entry, field validation, draft generation, document preview, PDF/DOCX export, saved document history, and administrator management of templates. It should also record generation status and errors so that failed operations can be diagnosed.

Non-functional requirements include usability, performance, security, availability, maintainability, scalability, and accessibility. Sensitive document information should be protected in transit and at rest. Authentication should use secure password hashing and session or token controls. Access checks should ensure that one user cannot retrieve another user's documents.

AI requirements include prompt construction, structured output, validation, refusal or fallback handling, and logging of model failures. AI output should never be accepted blindly. A validation layer should check required sections, placeholders, unsupported claims, and formatting before export. The system should clearly communicate that generated text is a draft and may require professional review.

Representative priorities are: authentication – Must; document templates – Must; form validation – Must; AI clause assistance – Must; PDF generation – Must; DOCX generation – Should; document history – Should; admin analytics – Could; multilingual support – Could.

Suggested System Flow

User → Web Interface → Authentication → Document Form → Validation → Template/AI Service → Output Validation → PDF/DOCX Renderer → Secure Storage → Download

3. Project Design

LegalEase can use a three-layer architecture: presentation layer, application/service layer, and data layer. The presentation layer contains responsive HTML/CSS/JavaScript or a modern frontend. The service layer contains authentication, validation, template processing, AI orchestration, document rendering, and API endpoints. The data layer contains users, templates, document instances, audit information, and configuration.

A typical generation sequence is: the browser sends validated form data to the backend; the backend authenticates the user; the template service identifies the selected document; the AI service receives a constrained prompt; the output is parsed; validation rules inspect the result; the document renderer builds a PDF or DOCX; the file is stored securely; and a download response is returned.

Suggested components include Frontend UI, REST API, Authentication Service, Template Service, AI Service, Validation Service, Document Renderer, Database, File Storage, and Audit Logger. A queue can be added later for large or slow generation jobs. This modular structure makes it easier to test and replace individual components.

Security design includes HTTPS, hashed passwords, authorization checks, input sanitization, file-type validation, rate limiting, secure secrets management, audit logging, and controlled retention. The application should minimize stored personal data and provide appropriate deletion and retention policies.

Suggested System Flow

User → Web Interface → Authentication → Document Form → Validation → Template/AI Service → Output Validation → PDF/DOCX
Renderer → Secure Storage → Download

4. Project Planning

Planning divides the work into milestones. Phase A covers requirements and wireframes. Phase B implements authentication and database models. Phase C develops templates and forms. Phase D integrates the AI service. Phase E implements PDF/DOCX generation. Phase F performs functional, security, usability, and performance testing. Phase G completes documentation and demonstration.

A practical schedule can assign one or two weeks to requirements and design, two weeks to backend and database development, two weeks to frontend and template work, one to two weeks to AI integration, one week to export and testing, and one week to final documentation and demonstration. The exact schedule depends on team size and academic deadlines.

Risks include incorrect AI output, prompt injection, exposure of personal data, invalid documents, service outages, high API cost, poor formatting, and template errors. Mitigations include deterministic validation, constrained prompts, access controls, secure storage, fallback templates, retries, monitoring, and human review.

Deliverables include source code, database schema, UI screens, architecture diagram, test cases, sample generated documents, installation instructions, user manual, administrator guide, and final presentation. Version control should be used throughout development so changes can be reviewed and reverted.

Suggested System Flow

User → Web Interface → Authentication → Document Form → Validation → Template/AI Service → Output Validation → PDF/DOCX Renderer → Secure Storage → Download

5. Development

Development can use Python Flask or FastAPI for the backend, HTML/CSS/JavaScript or React for the frontend, and SQLite/MySQL/PostgreSQL for persistence. ReportLab can be used for PDF creation, while python-docx can support DOCX output. The AI provider can be accessed through a protected server-side integration rather than exposing credentials in browser code.

Example backend structure: `app/routes`, `app/services`, `app/models`, `app/templates`, `app/validators`, `app/utils`, `app/static`, `app/tests`, `config.py`, `requirements.txt`, and `run.py`. The separation keeps business logic independent from page rendering and makes automated testing easier.

Example generation flow: receive JSON input, authenticate the user, validate required fields, generate a constrained draft, validate the result, render a PDF, and return it as a download. A real implementation should add authorization, error handling, secure temporary files, and audit logging.

AI prompt construction should provide the document type, approved template rules, user data, required sections, and output schema. The model should be instructed not to invent user facts. Structured JSON output can make validation easier. After generation, the server should validate the schema and required clauses before rendering.

Suggested System Flow

User → Web Interface → Authentication → Document Form → Validation → Template/AI Service → Output Validation → PDF/DOCX
Renderer → Secure Storage → Download

6. Testing

Testing verifies that LegalEase behaves correctly and safely. Unit tests should cover validation rules, template substitution, authentication helpers, document formatting functions, and AI-response parsing. Integration tests should cover the complete path from form submission to generated document. UI tests should check form navigation, errors, preview, and download.

Example test cases: an empty required name should be rejected; an invalid email should show an error; unauthorized document access should be denied; an unsupported document type should be rejected; malformed AI output should trigger a safe fallback; PDF generation should create a readable file; and a completed document should contain required headings and user-approved values.

Security testing should include authentication bypass attempts, broken access control, injection payloads, unsafe file uploads, rate-limit checks, session handling, and secret exposure checks. AI-specific tests should include prompt injection, conflicting instructions, missing context, unsupported legal claims, and attempts to make the system invent facts.

Acceptance criteria include successful document creation with valid inputs, clear errors for invalid inputs, protected private documents, stable PDF generation, correct formatting, and an explicit review disclaimer. Performance tests should measure response time and document generation under realistic concurrent use.

Suggested System Flow

User → Web Interface → Authentication → Document Form → Validation → Template/AI Service → Output Validation → PDF/DOCX Renderer → Secure Storage → Download

7. Documentation

Documentation should contain a project overview, objectives, scope, requirements, architecture, database design, API descriptions, installation steps, configuration instructions, testing results, security notes, user guide, administrator guide, maintenance procedure, and troubleshooting section. Diagrams should explain system flow and component relationships.

The user guide should explain sign-in, selecting a document, entering information, reviewing the draft, generating the final file, downloading it, and viewing saved documents. It should also explain that an AI-generated draft is not automatically legal advice and should be reviewed by an appropriate qualified professional when necessary.

Developer documentation should describe environment variables, dependency installation, database migration, API endpoints, AI configuration, file storage, logging, error handling, and deployment. Sensitive values such as API keys must never be committed to source control.

Maintenance documentation should define backup procedures, dependency updates, template review, model changes, security patching, log retention, and regression testing. A version number should be displayed so that generated documents and project releases can be traced to a known configuration.

Suggested System Flow

User → Web Interface → Authentication → Document Form → Validation → Template/AI Service → Output Validation → PDF/DOCX
Renderer → Secure Storage → Download

8. Demonstration

The final demonstration should begin with the problem statement and project objectives. Next, show the login or home screen, select a document type, complete a guided form, demonstrate validation, generate the AI-assisted draft, preview the result, and download a PDF. The presenter should explain which parts are deterministic templates and which parts are AI-assisted.

A recommended live sequence is: open LegalEase; select a sample document type; enter fictional demonstration data; submit the form; show the generation status; inspect the generated sections; show validation and disclaimer information; export PDF; open document history; and explain the architecture and security controls.

For an academic viva, explain why the project uses modular architecture, why validation is required after AI generation, how authentication protects documents, how templates are managed, how PDF files are created, and how testing reduces failures. Demonstrate one invalid input case and one successful generation case.

The conclusion is that LegalEase combines guided legal-document templates, validation, secure application design, and AI-assisted drafting into one academic software project. Future enhancements can include multilingual support, richer template libraries, retrieval-augmented legal knowledge, e-signature integration, analytics, and stronger compliance controls.

Suggested System Flow

User → Web Interface → Authentication → Document Form → Validation → Template/AI Service → Output Validation → PDF/DOCX
Renderer → Secure Storage → Download

Technical Appendix

Suggested Database Schema

Table	Important fields
users	id, name, email, password_hash, role, created_at
templates	id, name, version, document_type, template_body, active
documents	id, user_id, template_id, status, file_path, created_at
generations	id, document_id, model, prompt_version, status, created_at
audit_logs	id, user_id, action, resource_id, timestamp, metadata

Conclusion

LegalEase is designed as an academic and software-engineering demonstration for AI-assisted legal document drafting. Its modular architecture supports templates, validation, AI integration, secure storage, and document export while keeping human review central to real-world legal use.